

PD-2: Event categories

Status: in review · Revision 4 · Created by Ivan Blažević <ivan.blazevic@rubycode.co> · 30 September 2026

Overview

What

Organizers pick a category for each event: meetup, workshop, talk or conference. The events index shows the category on every card and can be filtered by it.

Why

The index mixes big conferences with small meetups, and people can't find the kind of event they want.

Where

The events index (/events), and the new and edit event forms.

Who

Ivan Blažević, Gather team

Background

Existing events should become meetups.

Existing Data Structure

Event (`app/models/event.rb` , **table** `events`)

Columns:

- `id` integer, not null
- `title` string, not null
- `description` text, nullable
- `starts_at` datetime, not null (indexed: `index_events_on_starts_at`)
- `venue` string, not null
- `capacity` integer, not null

- `organizer_id` integer, not null (indexed: `index_events_on_organizer_id` , foreign key to `users`)
- `created_at` , `updated_at` datetime, not null

Associations and behaviour this change touches:

- `belongs_to :organizer, class_name: "User"`
- `has_many :rsvps, has_many :attendees, through: :rsvps`
- `scope :upcoming, which is where(starts_at: Time.current..).order(:starts_at)` . The events index is built on this scope.
- `organized_by?(user)` . `EventsController#require_organizer` uses it to limit edit and update to the organizer.

Today there is no category, type or kind column on `events` .

User (`app/models/user.rb` , `table users`)

Columns: `id` , `email_address` (unique), `password_digest` , `name` (not null, default `""`), `created_at` , `updated_at` .

- `has_many :organized_events, class_name: "Event", foreign_key: :organizer_id` . `EventsController#create` builds events through this association.
- `has_many :sessions, has_many :rsvps` .

The index card shows the organizer's name. This change does not alter `users` .

Rsvp (`app/models/rsvp.rb` , `table rsvps`)

Columns: `id` , `event_id` , `user_id` , `status` (string, not null, check constraint `rsvps_status_check` limiting it to `going` or `waitlisted`), `created_at` , `updated_at` .

- `belongs_to :event, belongs_to :user`
- `enum :status, %w[ going waitlisted ].index_by(&:itself), validate: true` . This is the string-enum convention the new column will follow.

The index runs one grouped query, `Rsvp.going.where(event: @events).group(:event_id).count` , to show seats left. The filter must keep that query to exactly one. This change does not alter `rsvps` .

Session (`app/models/session.rb` , `table sessions`)

Not touched.

Architectural changes

Target design

Each event has exactly one category, stored as a string column `events.category` . The allowed values are a fixed list: `meetup` , `workshop` , `talk` , `conference` . There is no separate categories table, because custom categories and several categories per event are out of scope. The column

maps to a Rails string-backed enum on `Event`, the same way `Rsvp#status` does: `enum :category, %w[ meetup workshop talk conference ].index_by(&:itself), validate: true`.

Data flow

1. **Create and edit.** `app/views/events/_form.html.erb` gets a category field. `EventsController#event_params` permits `:category`. Create keeps its current shape (`Current.user.organized_events.build(event_params)`), and update keeps `require_organizer`.
2. **Index.** `EventsController#index` still starts from `Event.upcoming.includes(:organizer)`. If `params[:category]` is one of `Event.categories.keys`, it adds `.where(category: params[:category])`. Any other value, or no value, shows all categories. `@going_counts` stays one grouped query over the filtered `@events` relation.
3. **Display.** Each card in `app/views/events/index.html.erb` shows a category label. A filter bar above the list links to `/events` (All) and to `/events?category=meetup`, `?category=workshop`, and so on. These are plain GET links, so filtered pages can be bookmarked and shared, and no JavaScript is needed.

The controller keeps its current structure: no new controller, route, service object or job. `resources :events` in `config/routes.rb` already serves `/events` and accepts query parameters.

How legacy data coexists during rollout

- **Expand.** The column is added with the database default `'meetup'` and is nullable at first. From that point, any insert that doesn't mention `category` gets `meetup`. That covers app processes still running the old code during the deploy. On SQLite, existing rows read the default as soon as the column is added.
- **Dual write.** No old column is being replaced, so there is nothing to write twice. The database default is the safety net: old code writes nothing, and the database writes `meetup`.
- **Backfill.** A batched, idempotent task sets `category = 'meetup'` wherever it is `NULL`. This meets the requirement that existing events become meetups, and covers any row the default didn't reach.
- **Switch reads.** The new code, meaning the enum, form field, card label and filter, ships only after the column exists. It treats the category as always present. Until the backfill is confirmed, the card label helper shows nothing, rather than raising an error, if the category is ever `nil`.
- **Contract.** Once no nulls remain, a final migration makes the column `NOT NULL` and adds a check constraint for the four values. No column is removed or renamed, so `ignored_columns` is not needed.

Suggested ticket split

1. Migration: add the column and the index.
2. Backfill task.
3. Model enum, strong params and form field.
4. Card label and index filter.
5. Migration: NOT NULL and check constraint.

Ticket 3 depends on 1. Ticket 4 depends on 3. Ticket 5 depends on 2 and on the backfill having run in production.

Database changes

Categories live in their own `categories` table, and each event points to one of them through `events.category_id`. The allowed categories are rows in that table, not a list in code. There is no `events.category` string column and no enum on `Event`.

Step 1: Expand

MIGRATION `CREATECATEGORIES`

New table `categories`:

- `id` integer, primary key, not null
- `name` string, **not null**, no default. This is the display label, for example "Meetup".
- `slug` string, **not null**, no default. This is the stable key used in URLs and code, for example "meetup". The index filter reads it from `?category=workshop`.
- `color` string, **not null**, default `"#e11d48"`. The badge colour on event cards.
- `created_at`, `updated_at` datetime, not null (`t.timestamps`)
- Unique index `index_categories_on_slug` on `slug`. The index filter looks categories up by slug, and seeding relies on this index to skip rows that already exist.
- Unique index `index_categories_on_name` on `name`, so two categories can't have the same label.

```
class CreateCategories < ActiveRecord::Migration[8.1]
  def change
    create_table :categories do |t|
      t.string :name, null: false
      t.string :slug, null: false
      t.string :color, null: false, default: "#e11d48"

      t.timestamps

      t.index :slug, unique: true
      t.index :name, unique: true
    end
  end
end
```

MIGRATION `SEEDCATEGORIES` (REFERENCE DATA)

This migration inserts the four rows that production needs. It is a separate migration from the table creation, so each one has a single job.

slug	name
meetup	Meetup
workshop	Workshop
talk	Talk
conference	Conference

The migration defines its own small model class instead of loading `app/models/category.rb`. That way, later changes to the app model can't break an old migration. It uses `insert_all(..., unique_by: :slug)`, so running it again skips rows that are already there.

```
class SeedCategories < ActiveRecord::Migration[8.1]
  class MigrationCategory < ActiveRecord::Base
    self.table_name = "categories"
  end

  CATEGORIES = {
    "meetup" => "Meetup",
    "workshop" => "Workshop",
    "talk" => "Talk",
    "conference" => "Conference"
  }.freeze

  def up
    now = Time.current
    rows = CATEGORIES.map { |slug, name| { slug: slug, name: name, created_at: now, u
    MigrationCategory.insert_all(rows, unique_by: :slug)
  end

  def down
    MigrationCategory.where(slug: CATEGORIES.keys).delete_all
  end
end
```

`db/schema.rb` stores the structure only, not rows. So `bin/rails db:schema:load`, `db:prepare` on a new database, and the test database will **not** contain these four rows. Two things cover that:

- `db/seeds.rb` also creates them with `Category.find_or_create_by!(slug: ...)`, before the demo events, so the demo events can be given categories.
- The test suite creates categories through a `spec/factories/categories.rb` factory. It must never assume the migration's rows exist.

MIGRATION ADDCATEGORYTOEVENTS

On table `events` :

- Add column `category_id` : type `integer` (`add_reference`), **nullable**, **no default**. A default isn't possible because the `id` of the `meetup` row isn't guaranteed to be the same in every environment.
- Add foreign key `events.category_id → categories.id` , with no `on_delete` . The database then refuses to delete a category that events still use.
- Add composite index `index_events_on_category_id_and_starts_at` on `[:category_id, :starts_at]` . It serves the filtered index query (`WHERE category_id = ? AND starts_at >= ? ORDER BY starts_at`) and the foreign key lookups. For that reason, the single-column index that `add_reference` would normally create is left out, following the pattern in `CreateRsvps` . The existing `index_events_on_starts_at` keeps serving the unfiltered index.

```
class AddCategoryToEvents < ActiveRecord::Migration[8.1]
  def change
    # The [category_id, starts_at] index already leads with category_id.
    add_reference :events, :category, null: true, foreign_key: true, index: false
    add_index :events, [ :category_id, :starts_at ]
  end
end
```

This step only adds things. Old code never reads `category_id` , and any event it inserts gets `NULL` , which the column allows. On SQLite, adding a foreign key to an existing table may make Rails rebuild `events` , which `rsvps.event_id` references. Run all three migrations on a copy of production data first.

Step 2: Dual write (application code, no schema change)

No old column is being replaced, so nothing is written twice. "Dual write" here means that from this release on, every create and update path sets `category_id` :

- The form submits `category_id` .
- `Event` assigns the `meetup` category to a new record that has no category, if Outstanding question 3 is answered that way.

During the deploy, events created by processes still running the old code keep `category_id = NULL` . The backfill runs after this step so it also catches those rows.

Step 3: Backfill (data task, not a schema change)

Run this only after the step 2 code is live on every process.

The task is `events:backfill_category` in `lib/tasks/events.rake` :

1. Look up the meetup category with `Category.find_by!(slug: "meetup")` . If the row is missing, the task fails loudly instead of doing nothing.
2. Run `Event.where(category_id: nil).in_batches(of: 1_000).update_all(category_id: meetup.id)` .
3. Log the number of rows updated.

The task is idempotent and safe to re-run. It covers all events, past and upcoming (see Outstanding questions 1). `update_all` skips validations and callbacks and does not change `updated_at`.

Before step 5, confirm `Event.where(category_id: nil).count == 0` in production.

Step 4: Switch reads (application code, no schema change)

The card label, the filter (`params[:category] → Category.find_by(slug:) → where(category_id:)`) and the edit form read `category_id`. Until step 5 ships, the card label shows nothing if `category_id` is `NULL`, instead of raising an error.

Step 5: Contract (migration `EnforceCategoryOnEvents`)

Run this only after the check in step 3 returns zero.

- `change_column_null :events, :category_id, false`.
- The column still has no default. New rows get their category from the application, and the NOT NULL constraint rejects any path that forgets it.
- No check constraint is needed. The foreign key already limits values to existing `categories` rows.

```
class EnforceCategoryOnEvents < ActiveRecord::Migration[8.1]
  def change
    change_column_null :events, :category_id, false
  end
end
```

On SQLite, `change_column_null` rebuilds the `events` table. Rails' SQLite adapter handles this inside the migration, keeping the foreign keys and indexes. Take a backup first, run it on a copy of production data, and ship it separately from the feature code.

Removed or renamed columns

None. `ignored_columns` is not needed.

Resulting schema

```
create_table "categories", force: :cascade do |t|
  t.string "name", null: false
  t.string "slug", null: false
  t.string "color", null: false, default: "#e11d48"
  t.datetime "created_at", null: false
  t.datetime "updated_at", null: false
  t.index ["name"], name: "index_categories_on_name", unique: true
  t.index ["slug"], name: "index_categories_on_slug", unique: true
end

# events gains:
```

```
t.integer "category_id", null: false
t.index ["category_id", "starts_at"], name: "index_events_on_category_id_and_starts_at"

add_foreign_key "events", "categories"
```

Application changes

Categories are records in the new `categories` table. Each event points to one through `events.category_id`. There is no enum on `Event`, and no list of categories in application code. The display name and badge colour come from the `Category` row, and the `slug` is the stable key used in URLs.

Model: `app/models/category.rb` (new)

```
class Category < ApplicationRecord
  has_many :events, dependent: :restrict_with_error

  validates :name, presence: true, uniqueness: true
  validates :slug, presence: true, uniqueness: true, format: { with: /\A[a-z0-9-]+\z/ }
  validates :color, format: { with: /\A#\h{6}\z/ }

  scope :ordered, -> { order(:name) }
end
```

- `dependent: :restrict_with_error` matches the foreign key, which has no `on_delete`. Destroying a category that events still use adds a validation error instead of raising a database error.
- The `color` format check matters because the colour is written into a `style` attribute on the card (see Helper). Only `#rrggbb` values are accepted.
- `ordered` sorts alphabetically: Conference, Meetup, Talk, Workshop. There is no `position` column, and ordering by `id` depends on insert order in each environment. If the team wants the brief's order (meetup, workshop, talk, conference), that needs a `position` column, which Database changes does not include. See Outstanding questions.
- No controller, route or admin screen manages categories in this change. They are changed only through migrations or the console.

Model: `app/models/event.rb`

- Add `belongs_to :category, optional: true` for the rollout. Rails requires `belongs_to` by default, which would make legacy rows with `category_id = NULL` invalid on any update before the backfill. The contract ticket (`EnforceCategoryOnEvents`) also removes `optional: true`, so the model then requires a category, like the database does.
- Add `validates :category, presence: true, if: :category_id?`. With `optional: true`, a `category_id` that points to no row would otherwise reach the foreign key and raise

`ActiveRecord::InvalidForeignKey`, which becomes a 500. This validation turns it into a normal form error ("Category can't be blank") and a 422.

- New records get their category from one of two options, depending on Outstanding question 3:
- **Preselect Meetup:** add `before_validation :assign_default_category, on: :create`. It sets `self.category ||= Category.find_by(slug: "meetup")`.
- **Organizer must choose:** add `validates :category, presence: true, on: :create` instead.

Either way, every event created from this release onward has a `category_id`. That is the dual-write step.

- Add `scope :in_category, ->(category) { where(category:) }` so the controller filter reads clearly and model specs can test it with `upcoming`.
- Remove the enum wording from the earlier plan. There are no `Event.meetup` scopes, no `meetup?` predicates and no `Event.categories`.

Controller: `app/controllers/events_controller.rb`

- `index`:

```
def index
  @categories = Category.ordered
  @category = @categories.find { |category| category.slug == params[:category] }
  @events = Event.upcoming.includes(:organizer, :category)
  @events = @events.in_category(@category) if @category
  @going_counts = Rsvp.going.where(event: @events).group(:event_id).count
end
```

- The filter bar needs the full category list anyway, so the `slug` from `params[:category]` is matched against that loaded list. This avoids a separate `Category.find_by(slug:)` query. A missing or unknown slug leaves `@category` as `nil` and shows All. The raw parameter is never passed to a query and never rendered.
- `includes(:category)` loads the badge name and colour for every card in one query.
- `@going_counts` stays a single grouped query over the filtered `@events` relation.
- `new`: if the answer to Outstanding question 3 is "preselect Meetup", build with `category: Category.find_by(slug: "meetup")` so the select shows Meetup. Otherwise, no change.
- `new`, `create`, `edit` and `update` re-render the form, which needs the category list. The form reads `Category.ordered` directly (see Views). No extra instance variable or `before_action` is needed.
- `event_params`: `params.expect(event: %i[ title description starts_at venue capacity category_id ])`.
- `create`, `update`, `show`: no logic changes. `create` keeps `Current.user.organized_events.build(event_params)`. `require_organizer` still guards `edit` and `update`, so only the organizer can change `category_id`.

Helper: `app/helpers/application_helper.rb`

- Add `event_category_badge(event)` :

```
def event_category_badge(event)
  return if event.category.nil?

  tag.span event.category.name, class: "category-badge", style: "--category-color: #{
end
```

- It returns `nil` while legacy rows still have `category_id = NULL` , so the card renders without a badge instead of raising an error. After the contract step, this guard can be removed.
- The colour goes into a CSS custom property. The stylesheet decides how to use it, such as for the background, border or dot. `tag.span` escapes attribute values, and `Category` validates the `#rrggbb` format.
- Drop `event_category_name` and the i18n labels from the earlier plan. The label is `Category#name` .

Views

- `app/views/events/_form.html.erb` :
- Add a `Category` field after `Title`, using `form.collection_select :category_id, Category.ordered, :id, :name, { prompt: "Choose a category" }, required: true` . `form_with` model: `event` preselects the saved value on edit.
- A legacy event with no category shows the prompt on edit. If the organizer saves without choosing, `category_id` stays `NULL` until the backfill, which `optional: true` allows.
- Whether this is a select or a radio group is still Outstanding question 4. The existing error block already shows the new validation messages.
- `app/views/events/index.html.erb` :
- In each `.event-card__body` , render `event_category_badge(event)` in its own element above or next to `.event-card__title` . The "Organized by ... · N of N seats left" line does not change.
- Add `<nav class="category-filter" aria-label="Filter by category">` between `.hero` and `.event-list` . It has an "All" link to `events_path` and one link per category in `@categories` to `events_path(category: category.slug)` . The active link gets `aria-current="page"` , which is "All" when `@category` is `nil` .
- Filtered empty state: "No upcoming `#{@category.name.downcase.pluralize}` yet." plus a link to All. Keep the current "No upcoming events yet." when no filter is set.
- The hero copy change is still Outstanding question 7.
- `app/views/events/show.html.erb` : no change unless Outstanding question 6 adds the badge to `#event-facts` . If it does, reuse `event_category_badge` .

Styles: `app/assets/stylesheets/application.css`

- `.category-badge` uses `var(--category-color)` , for example as the background with white text, or as a coloured border and dot on a neutral background.

- Add `.category-filter` and its `[aria-current="page"]` active state.
- There are no per-category modifier classes, because colours come from the database.
- Every seeded category gets the column default `#e11d48` unless someone sets distinct colours, and the text must stay readable on any valid colour. See Outstanding questions.

Backfill task: `lib/tasks/events.rake` (new file)

`events:backfill_category`, as described under Database changes:

- `meetup = Category.find_by!(slug: "meetup")`
- `Event.where(category_id: nil).in_batches(of: 1_000).update_all(category_id: meetup.id)`
- Log the number of rows updated.

Seeds: `db/seeds.rb`

- Before the demo events, create the four categories with `Category.find_or_create_by!(slug: { |c| c.name = ... })`. Seeds may also set a `color` for each so the demo shows different badges.
- Add a `category:` to each demo event hash. "Ruby Zagreb Meetup #42" is `meetup`, and "Hotwire workshop: Turbo Streams in practice" is `workshop`. "Rails upgrade clinic" should also be assigned one, for example `workshop`.

Test support

- `spec/factories/categories.rb` (new):
- `sequence(:slug), name { slug.titleize }, color { "#e11d48" }`.
- `initialize_with { Category.find_or_initialize_by(slug:) }`, so many events can share the `meetup` category without breaking the unique index on `slug`.
- `spec/factories/events.rb`:
- Add `category { association :category, slug: "meetup" }`.
- Add traits `:workshop`, `:talk` and `:conference`, each with the matching slug.
- Add a `:legacy` trait with `category { nil }` for rows from before the backfill.
- `features/step_definitions/events_index_steps.rb`: add steps to create an event in a named category and to click a filter link. The existing step `{int} query/queries ran against {word} stays`, and the filtered index keeps exactly one query against `rsvps`.

Policies and jobs

The app has no policy layer (there is no `app/policies`). Authorization stays in

`EventsController#require_organizer`, and `allow_unauthenticated_access` only: `%i[ index show ]` keeps the filtered index public. No jobs are added.

Infrastructure changes

There are no new queues, background jobs, external services or environment variables.

There is no feature flag. The Gemfile has no flag library, the change is small, and the category field and filter can ship directly. If the team wants to hide the filter until categories have been assigned to real events, that needs a decision (see Outstanding questions).

Rollout order:

1. Deploy migration `AddCategoryToEvents` (column with default `'meetup'`, plus the index). Old code keeps running.
2. Run `bin/rails events:backfill_category` in production and confirm `Event.where(category: nil).count == 0`.
3. Deploy the application code: enum, strong params, form field, card label and filter. This can go in the same release as step 1, as long as the deploy runs migrations before new processes serve traffic.
4. Once organizers have had time to recategorize their events, deploy migration `EnforceCategoryOnEvents` (NOT NULL plus check constraint). Take a database backup first, because on SQLite this rebuilds the `events` table.

Work overview

Out of Scope

Custom categories, several categories per event, and search.

Risks

- **Every existing event shows up as a meetup, including conferences and workshops.** The team asked for this, but the new filter will look wrong until organizers fix their events. *Mitigation:* tell organizers before release. Optionally give the team a one-off list of upcoming events so they can ask organizers to recategorize. Whether an admin should correct them in bulk is an Outstanding question.
- **Old code running during the deploy creates events without a category.** *Mitigation:* the column default `'meetup'` fills the value, and the column stays nullable until step 4.
- **The SQLite table rebuild in the contract migration.** `events` is referenced by `rsvps.event_id`, and both `change_column_null` and `add_check_constraint` rebuild the table on SQLite. *Mitigation:* run the migration on a copy of production data first, take a backup before deploying, and ship it separately from the feature code.
- **An invalid category from a tampered form or a bad query string.** *Mitigation:* `enum ... validate: true` turns bad form values into a 422 with an error message. The index ignores filter values not in `Event.categories.keys`, so it never raises and never runs a query with arbitrary input.
- **The filter adds extra queries per card, or breaks the one-query seat counts.** *Mitigation:* the filter is a `where` on the same relation. `@going_counts` stays a single grouped query, and the existing request spec that asserts one `rsvps` query is extended to cover filtered pages.

- **Enum-generated methods clash with future code** (for example a future `talk` association).
Mitigation: none of the generated methods clash today. If the team wants extra safety, use `prefix: :category ( category_meetup? )` and decide before tickets start (see Outstanding questions).
- **Cucumber and request specs that assert exact card text fail when the card gains a label.**
Mitigation: the category label goes in its own element, and assertions like "Organized by Ana Kovač · 3 of 3 seats left" in `features/events.feature` keep matching because that line doesn't change.

Performance

- **Index query count.** It stays the same as today: one query for events (with or without the category condition), one for organizers from `includes(:organizer)`, and one grouped `rsvps` count. The card label reads a column already loaded on each event, so there is no N+1.
- **Indexes.** The new composite index `[category, starts_at]` serves `WHERE category = ? AND starts_at >= ? ORDER BY starts_at` without a separate sort. The unfiltered page keeps using `index_events_on_starts_at`. Category has only four values, so an index on `category` alone would help little and is not added.
- **Table size.** The number of rows in `events` is unknown but expected to be small for a community site. Adding a column with a constant default on SQLite does not rewrite existing rows. The contract migration does rebuild the table, and its duration grows with row count (see Risks).
- **Backfill.** `in_batches(of: 1_000).update_all(...)` over rows where `category IS NULL`. Each batch is one UPDATE, validations and callbacks are skipped, and the task can be re-run safely. At the expected table size it should finish in seconds.
- **Filter links.** They are built from the constant list of categories. No count per category is shown, so rendering the filter bar costs no queries. If counts per category are added later, use a single `group(:category).count`.

Security

- **Authorization.** Unchanged. Anyone, including guests, can view and filter `/events` (`allow_unauthenticated_access only: %i[ index show ]`). Creating an event requires sign-in. Only the organizer can edit or update an event, through `require_organizer`, which now also covers changing the category. A request spec should confirm that a non-organizer cannot change the category with a PATCH.
- **Input validation.**
 - `category` is added to the `params.expect allow-list`.
 - The model enum with `validate: true` rejects any value outside the four allowed ones.
 - The step 4 check constraint `events_category_check` enforces the same list in the database.
 - The `category` query parameter on the index is checked against `Event.categories.keys` before use. Unknown values are ignored, not echoed back into the page.
- **Data exposure.** Category is public and non-sensitive, like the title and venue. The index cards expose no new user data.
- **Output.** Labels come from a fixed list and are rendered through normal ERB escaping, so there is no raw HTML.

Risk Level: LOW. The change adds a public, allow-listed attribute behind the existing organizer check and exposes no personal data.

Monitoring

- **Errors.** Watch for `ArgumentError` or `ActiveRecord::StatementInvalid` from `EventsController#index`, `#create` and `#update` in the first days. Any `events_category_check` violation after step 4 means some code path is writing an invalid value.
- **Request outcomes.** Compare the 422 rate on `POST /events` and `PATCH /events/:id` with the rate before release. A jump suggests the category field is confusing or missing from a form.
- **Backfill.** Record the row count logged by `events:backfill_category`. Before shipping step 4, confirm `Event.where(category: nil).count == 0` in a production console.
- **Adoption.** Periodically run `Event.group(:category).count` and the same over `Event.upcoming`. If almost every new event is still `meetup` weeks after release, the preselected default may be hiding the choice.
- **Response time.** Watch the response time of `GET /events` with and without `category`. It should not change noticeably.
- **Jobs.** None are added, so there is nothing to watch.

Outstanding questions

1. **Scope of the backfill (before step 2).** Should all existing events become meetups, including past ones, or only upcoming ones? This plan assumes all of them.
2. **Known non-meetups (before step 2).** Will someone correct existing events that are clearly conferences, workshops or talks, or is that left to each organizer?
3. **Default in the form (before step 3).** Should the new-event form preselect Meetup, or require organizers to choose, with a blank prompt and no model default? This decides whether the column default stays after step 4.
4. **Form control (before step 3).** Select menu or radio buttons for the category?
5. **Wording (before step 3).** Are the display labels "Meetup", "Workshop", "Talk", "Conference", in English only? Should they live in `config/locales/en.yml`?
6. **Show page (before step 3).** Should the category also appear on the event show page (`app/views/events/show.html.erb`)? The brief only mentions the index and the forms.
7. **Hero copy (before step 3).** Should the index hero text "Meetups, workshops and talks from the Zagreb tech community." change to mention conferences?
8. **Filter bar details (before step 3).** Should the bar show counts per category? Should it keep the chosen filter when the user navigates back from an event?
9. **Enum method names (before step 3).** Plain enum methods (`Event.talk`, `event.meetup?`) or prefixed ones (`category_talk?`)?
10. **Deploy pipeline (before step 1).** Does the production deploy run `db:migrate` or `db:prepare` before new app processes serve traffic? This decides whether steps 1 and 3 can share a release.
11. **Timing of the contract step (before step 4).** How long should organizers have to recategorize before the NOT NULL and check-constraint migration ships? Is a maintenance window needed for the SQLite table rebuild?

Testing

Model: `spec/models/event_spec.rb`

- Accepts each of `meetup`, `workshop`, `talk` and `conference`.
- Is invalid with an unknown category (for example `"party"`) and shows an error on `:category`. It does not raise.
- Is invalid with `category: nil`.
- A new `Event` defaults to `meetup` when no category is given.
- `Event.workshop` and similar scopes return only events of that category. Combined with `upcoming`, the result stays ordered by `starts_at`.

Backfill task

- Sets `meetup` on rows where `category` is `NULL` (created with `update_column(:category, nil)` before the contract migration).
- Leaves rows that already have a category unchanged.
- Running it twice changes nothing and does not touch `updated_at`.

Migrations

- `db/schema.rb` shows `category` with default `'meetup'` and the `[category, starts_at]` index. After step 4, it also shows `null: false` and `events_category_check`.
- After step 4, inserting an invalid category through SQL fails.

Request specs: `spec/requests/events_spec.rb`

- The index shows the category label on every card, and a legacy event shows "Meetup".
- `GET /events?category=workshop` lists only upcoming workshops and marks the Workshop filter link as active.
- `GET /events?category=bogus` and `GET /events` list every upcoming category without error.
- A filtered index with no matches shows the category-specific empty state and a link to All.
- Guests can view and filter the index.
- The filtered index still loads going counts in a single `rsvps` query (extend the existing query-count spec).
- A signed-in user can create an event with `category: "talk"`, and it is saved.
- Creating with an invalid category returns 422 and re-renders the form with an error.
- The organizer can change the category through PATCH.
- A non-organizer's PATCH to `category` leaves it unchanged.
- Guests are still redirected away from `new`.

Helper: `spec/helpers/application_helper_spec.rb` **(new)**

- `event_category_label` renders the human label and CSS modifier for each category.
- It returns nothing for a blank category.

System and Cucumber: `features/events.feature` or a new

`features/event_categories.feature`

- A guest sees category labels on the cards and narrows the list by clicking "Workshop", then returns with "All".
- An organizer creates an event, picks "Conference", and sees it labelled Conference on the index.
- An organizer edits an existing event, previously a legacy meetup, changes it to "Talk", and it appears under the Talk filter.
- Someone who is not the organizer still sees no "Edit event" link, so has no way to change the category.
- The existing scenario "Organized by Ana Kovač · 3 of 3 seats left" still passes.

Sign-off

Role	Decision	By	When
Review	pending		